

**Московский государственный технический университет
им. Н.Э. Баумана**

УТВЕРЖДАЮ:

Большаков С.А.

"__" _____ 2026 г.

Курсовая работа по курсу «Системное программирование»

Исходный текст программного продукта

(вид документа)

писчая бумага

(вид носителя)

64

(количество листов)

ИСПОЛНИТЕЛИ:

студент группы ИУ5-42Б

Щепнов Ф. В.,

"__" _____ 2026 г.

Содержание

1. Файл tsr.ls3

1. Файл tsr.ls


```

47
48      0178  00      signaturePrintingEnabled
DB      0      ; флаг функции вывода      +
49      информации об авторе
50      0179  00      cursiveEnabled      DB
      0      ; флаг перевода символа в курсив
51
52
53      017A  00      cursiveSymbol      DB      00000000b ;
символ, составленный из единичек (его курсивный вариант)
54      017B  00      DB
      00000000b
55      017C  C3      DB
      11000011b
56      017D  C3      DB
      11000011b
57      017E  C3      DB
      11000011b

```

tsr.asm

58	017F	C3			DB
	11000011b				
59	0180	C3			DB
	11000011b				
60	0181	7F			DB
	01111111b				
61	0182	06			DB
	00000110b				
62	0183	0C			DB
	00001100b				
63	0184	18			DB
	00011000b				
64	0185	30			DB
	00110000b				
65	0186	60			DB
	01100000b				
66	0187	00			DB
	00000000b				
67	0188	00			DB
	00000000b				
68	0189	00			DB
	00000000b				
69					
70	018A	97	charToCursiveIndex		DB
	'Ч' ; символ для замены				
71	018B	10*(FF)	savedSymbol		
DB	16 dup(0FFh) ; переменная для		+		
72			хранения старого символа		
73					
74		=00FF	true		
		equ 0FFh ; +			

```

75                                     константа истинности
76 019B ????                          old_int9hOffset
DW ?      ; адрес старого      +
77                                     обработчика int 9h
78 019D ????                          old_int9hSegment
DW ?      ; сегмент старого      +
79                                     обработчика int 9h
80 019F ????                          old_int1ChOffset
DW ?      ; адрес старого      +
81                                     обработчика int 1Ch
82 01A1 ????                          old_int1ChSegment
DW ?      ; сегмент старого обработчика +
83                                     int 1Ch
84 01A3 ????                          old_int2FhOffset
DW ?      ; адрес старого      +
85                                     обработчика int 2Fh
86 01A5 ????                          old_int2FhSegment
DW ?      ; сегмент старого обработчика +
87                                     int 2Fh
88
89 01A7 00                                unloadTSR
DB      0 ; 1 - выгрузить      +
90                                     резидент
91 01A8 00                                notLoadTSR
DB 0      ; 1 - не загружать
92 01A9 0000                            counter
DW 0
93 =0005                                printDelay
equ 5 ; задержка перед выводом +
94                                     "подписи" в секундах
95 01AB 0002                            printPos
DW      2;    положение подписи на+
96                                     экране. 0 - верх, 1 - центр, 2 - низ
97

```



```

98                                     ;@      заменить на собственные
данные.      формирование таблицы идет по строке большей длины  +
99                                     (1я строка).
100    01AD B3 99 A5 AF AD AE      A2+      signatureLine1
      DB      179, 'Щепнов Филипп      +
101      20 94 A8 AB A8 AF      AF+  ',      179
102      20 20 20 20 20 20      20+
103      20 20 20 20 20 20      20+
104      20 20 20 20 20 20      20+
105      20 20 20 20 20 20      20+
106      20 20 20 20 20 20      20+
107      20 20 20 20 20 20      20+
108      B3
109      =0039      Line1_length
      equ      $-signatureLine1
110    01E6 B3 88 93 35 2D 34      32+      signatureLine2
      DB      179, 'ИУ5-42Б      +
111      81 20 20 20 20 20      20+  ',      179
112      20 20 20 20 20 20      20+
113      20 20 20 20 20 20      20+
114      20 20 20 20 20 20      20+

```

tsr.asm

```

115      20 20 20 20 20 20      20+
116      20 20 20 20 20 20      20+
117      20 20 20 20 20 20      20+
118      B3
119      =0039                      Line2_length
      equ      $-signatureLine2
120      021F B3 82 A0 E0 A8 A0      AD+      signatureLine3
      DB      179, 'Вариант #25          +
121      E2 20 23 32 35 20      20+      ', 179
122      20 20 20 20 20 20      20+
123      20 20 20 20 20 20      20+
124      20 20 20 20 20 20      20+
125      20 20 20 20 20 20      20+
126      20 20 20 20 20 20      20+
127      20 20 20 20 20 20      20+
128      B3
129      =0039                      Line3_length
      equ      $-signatureLine3
130      0258 3E 74 73 72 2E 63      6F+      helpMsg DB '>tsr.com
[/?] ', 10, 13
131      6D 20 5B 2F 3F 5D      20+
132      0A 0D
133      0268 20 5B 2F 3F 5D 20      2D+      DB
      ' [/?] - вывод данной справки', 10, 13
134      20 A2 EB A2 AE A4      20+
135      A4 A0 AD AD AE A9      20+
136      E1 AF E0 A0 A2 AA      A8+
137      0A 0D

```

138	0286	20 20 54 53 52 2E	43+	DB
'	TSR.COM - выгрузка резидента из памяти', 10, 13			
139		4F 4D 20 2D 20 A2	EB+	
140		A3 E0 E3 A7 AA A0	20+	
141		E0 A5 A7 A8 A4 A5	AD+	
142		E2 A0 20 A8 A7 20	AF+	
143		A0 AC EF E2 A8 0A	0D	
144	02B0	20 20 46 35 20 20	2D+	DB
'	F5 - вывод ФИО и группы по таймеру в низ +			
145		20 A2 EB A2 AE A4	20+ экрана', 10, 13	
146		94 88 8E 20 A8 20	A3+	
147		E0 E3 AF AF EB 20	AF+	
148		AE 20 E2 A0 A9 AC	A5+	
149		E0 E3 20 A2 20 AD	A8+	
150		A7 20 ED AA E0 A0	AD+	
151		A0 0A 0D		
152	02E4	20 20 46 36 20 20	2D+	DB
'	F6 - включение и отключение курсивного вывода +			
153		20 A2 AA AB EE E7	A5+ русского символа Ч', 10, 13	
154		AD A8 A5 20 A8 20	AE+	
155		E2 AA AB EE E7 A5	AD+	
156		A8 A5 20 AA E3 E0	E1+	
157		A8 A2 AD AE A3 AE	20+	
158		A2 EB A2 AE A4 A0	20+	
159		E0 E3 E1 E1 AA AE	A3+	
160		AE 20 E1 A8 AC A2	AE+	
161		AB A0 20 97 0A 0D		
162	0329	20 20 46 37 20 20	2D+	DB
'	F7 - включение и отключение частичной +			
163		20 A2 AA AB EE E7	A5+ русификации клавиатуры([, W,	
X, I, O -> ХЦЧШЩ)', 10, 13				
164		AD A8 A5 20 A8 20	AE+	
165		E2 AA AB EE E7 A5	AD+	

166	A8 A5 20 E7 A0 E1	E2+
167	A8 E7 AD AE A9 20	E0+
168	E3 E1 A8 E4 A8 AA	A0+
169	E6 A8 A8 20 AA AB	A0+
170	A2 A8 A0 E2 E3 E0	EB+
171	28 5B 2C 20 57 2C	20+

tsr.asm

```

172          58 2C 20 49 2C 20      4F+
173          20 2D 3E 20 95 96      97+
174          98 99 29 0A 0D
175 0382 20 20 46 38 20 20      2D+                                DB
' F8 - включение и отключение режима ограничения +
176          20 A2 AA AB EE E7      A5+ ввода только латинскими
буквами', 10, 13
177          AD A8 A5 20 A8 20      AE+
178          E2 AA AB EE E7 A5      AD+
179          A8 A5 20 E0 A5 A6      A8+
180          AC A0 20 AE A3 E0      A0+
181          AD A8 E7 A5 AD A8      EF+
182          20 A2 A2 AE A4 A0      20+
183          E2 AE AB EC AA AE      20+
184          AB A0 E2 A8 AD E1      AA+
185          A8 AC A8 20 A1 E3      AA+
186          A2 A0 AC A8 0A 0D
187
188          =017D                                helpMsg_length
equ $-helpMsg
189 03D5 8E E8 A8 A1 AA A0      20+                                errorParamMsg
DB 'Ошибка параметров командной +
190          AF A0 E0 A0 AC A5      E2+ строки', 10, 13
191          E0 AE A2 20 AA AE      AC+
192          AC A0 AD A4 AD AE      A9+
193          20 E1 E2 E0 AE AA      A8+
194          0A 0D

```

```

195          =0025                      errorParamMsg_length    equ
$-errorParamMsg

196

197    03FA  DA 37*(C4) BF                tableTop
      DB      218, Line1_length-2 dup+

198                      (196), 191

199          =0039                      tableTop_length
equ    $-tableTop

200    0433  C0 37*(C4) D9                tableBottom
DB      192, Line1_length-2 dup (196), +

201                      217

202          =0039                      tableBottom_length
equ    $-tableBottom

203

204                      ; сообщения

205    046C  90 A5 A7 A8 A4 A5  AD+        installedMsg
      DB      'Резидент загружен!$'

206          E2 20 A7 A0 A3 E0  E3+

207          A6 A5 AD 21 24

208    047F  90 A5 A7 A8 A4 A5  AD+        alreadyInstalledMsg
DB      'Резидент уже загружен$'

209          E2 20 E3 A6 A5 20  A7+

210          A0 A3 E0 E3 A6 A5  AD+

211          24

212    0495  8D A5 A4 AE E1 E2  A0+        noMemMsg
      DB      'Недостаточно памяти$'

213          E2 AE E7 AD AE 20  AF+

214          A0 AC EF E2 A8 24

215    04A9  8D A5 20 E3 A4 A0  AB+        notInstalledMsg
DB      'Не удалось загрузить резидент$'

216          AE E1 EC 20 A7 A0  A3+

217          E0 E3 A7 A8 E2 EC  20+

218          E0 A5 A7 A8 A4 A5  AD+

219          E2 24

```

```

220
221    04C7  90 A5 A7 A8 A4 A5    AD+      removedMsg
          DB      'Резидент выгружен'
222          E2 20 A2 EB A3 E0    E3+
223          A6 A5 AD
224          =0011                removedMsg_length
equ    $-removedMsg
225
226    04D8  8D A5 20 E3 A4 A0    AB+      noRemoveMsg
          DB      'Не удалось выгрузить резидент'
227          AE E1 EC 20 A2 EB    A3+
228          E0 E3 A7 A8 E2 EC    20+

```

tsr.asm

```

229          E0 A5 A7 A8 A4 A5      AD+
230          E2
231          =001D                    noRemoveMsg_length
equ    $-noRemoveMsg
232
233    04F5  46 35                    f1_txt
      DB      'F5'
234    04F7  46 36                    f2_txt
      DB      'F6'
235    04F9  46 37                    f3_txt
      DB      'F7'
236    04FB  46 38                    f4_txt
      DB      'F8'
237          =0002                    fx_length
equ    $-f4_txt
238
239    04FD                                changeFx proc
240    04FD  50                            push AX
241    04FE  53                            push BX
242    04FF  51                            push CX
243    0500  52                            push DX
244    0501  55                            push BP
245    0502  06                            push ES
246    0503  33 DB                        xor BX, BX
247
248    0505  B4 03                        mov AH, 03h
249    0507  CD 10                        int 10h
250    0509  52                            push DX

```



```

251
252    050A  0E                                push CS
253    050B  07                                pop  ES
254
255    050C                                _checkF5:
256    050C  BD 04F5r                          lea BP, f1_txt
257    050F  B9 0002                          mov CX, fx_length
258    0512  B7 00                            mov BH, 0
259    0514  B6 00                            mov DH, 0
260    0516  B2 4E                            mov DL, 78
261    0518  B8 1301                          mov AX, 1301h
262
263    051B  80 3E 0178r FF                    cmp
signaturePrintingEnabled, true
264    0520  74 07                            je    _greenF5
265
266    0522                                _redF5:
267    0522  B3 4F                            mov BL, 01001111b ;
red
268    0524  CD 10                            int 10h
269    0526  EB 08 90                          jmp _checkF6
270
271    0529                                _greenF5:
272    0529  BD 04F5r                          lea BP, f1_txt
273    052C  B3 2F                            mov BL, 00101111b ;
green
274    052E  CD 10                            int 10h
275
276    0530                                _checkF6:
277    0530  BD 04F7r                          lea BP, f2_txt
278    0533  B9 0002                          mov CX, fx_length
279    0536  B7 00                            mov BH, 0

```

280	0538	B6 01	mov DH, 1
281	053A	B2 4E	mov DL, 78
282	053C	B8 1301	mov AX, 1301h
283			
284	053F	80 3E 0179r FF	cmp cursiveEnabled, true
285	0544	74 49	je _greenF8

tsr.asm

```

286
287      0546                      _redF6:
288      0546  B3 4F                      mov BL, 01001111b ;
red
289      0548  CD 10                      int 10h
290      054A  EB 05 90                    jmp _checkF7
291
292      054D                      _greenF6:
293      054D  B3 2F                      mov BL, 00101111b ;
green
294      054F  CD 10                      int 10h
295
296      0551                      _checkF7:
297      0551  BD 04F9r                    lea BP, f3_txt
298      0554  B9 0002                    mov CX, fx_length
299      0557  B7 00                      mov BH, 0
300      0559  B6 02                      mov DH, 2
301      055B  B2 4E                      mov DL, 78
302      055D  B8 1301                    mov AX, 1301h
303
304      0560  80 3E 0177r FF              cmp translateEnabled,
true
305      0565  74 07                      je    _greenF7
306
307      0567                      _redF7:
308      0567  B3 4F                      mov BL, 01001111b ;
red
309      0569  CD 10                      int 10h

```

```

310    056B EB 05 90                                jmp _checkF8
311
312    056E                                _greenF7:
313    056E B3 2F                                mov BL, 00101111b ;
green
314    0570 CD 10                                int 10h
315
316    0572                                _checkF8:
317    0572 BD 04FB r                             lea BP, f4_txt
318    0575 B9 0002                             mov CX, fx_length
319    0578 B7 00                             mov BH, 0
320    057A B6 03                             mov DH, 3
321    057C B2 4E                             mov DL, 78
322    057E B8 1301                             mov AX, 1301h
323
324    0581 80 3E 016Cr FF                     cmp ignoreEnabled,
true
325    0586 74 07                                je _greenF8
326
327    0588                                _redF8:
328    0588 B3 4F                                mov BL, 01001111b ;
red
329    058A CD 10                                int 10h
330    058C EB 05 90                                jmp _outFx
331
332    058F                                _greenF8:
333    058F B3 2F                                mov BL, 00101111b ;
green
334    0591 CD 10                                int 10h
335
336    0593                                _outFx:
337    0593 5A                                pop DX
338    0594 B4 02                                mov AH, 02h

```

339	0596	CD 10	int 10h
340			
341	0598	07	pop ES
342	0599	5D	pop BP

tsr.asm

```

343      059A  5A                                pop  DX
344      059B  59                                pop  CX
345      059C  5B                                pop  BX
346      059D  58                                pop  AX
347      059E  C3                                ret
348      059F                                changeFx endp
349
350                                ;новый   обработчик
351      059F                                new_int9h proc far
352                                ; сохраняем значения всех,
      изменяемых регистров в стеке
353      059F  56                                push SI
354      05A0  50                                push AX
355      05A1  53                                push BX
356      05A2  51                                push CX
357      05A3  52                                push DX
358      05A4  06                                push ES
359      05A5  1E                                push DS
360                                ; синхронизируем CS и DS
361      05A6  0E                                push CS
362      05A7  1F                                pop   DS
363
364      05A8  B8 0040                                mov   AX, 40h ; 40h-
      сегмент, где хранятся флаги сост-я клавиатуры, кольцо.      +
365                                буфер ввода
366      05AB  8E C0                                mov   ES, AX

```

```

367    05AD E4 60                                in      AL, 60h ;
записываем в AL скан-код нажатой клавиши
368
369
370
371
372                                ;@    далее -    код для
    всех вариантов
373
374                                ;проверка F5-F8
375    05AF                                _test_Fx:
376    05AF 2C 3A                                sub AL, 58 ; в AL теперь
номер функциональной клавиши
377    05B1                                _F5:
378    05B1 3C 05                                cmp AL, 5 ; F5
379    05B3 75 0A                                jne _F6
380    05B5 F6 16 0178r                        not
signaturePrintingEnabled
381    05B9 E8 FF41                                call changeFx
382    05BC EB 2E 90                                jmp
_translate_or_ignore
383    05BF                                _F6:
384    05BF 3C 06                                cmp AL, 6 ; F6
385    05C1 75 0D                                jne _F7
386    05C3 F6 16 0179r                        not cursiveEnabled
387    05C7 E8 FF33                                call changeFx
388    05CA E8 01EF                                call setCursive ;
перевод символа в курсив и обратно в зависимости от +
389                                флага cursiveEnabled
390    05CD EB 1D 90                                jmp
_translate_or_ignore
391    05D0                                _F7:
392    05D0 3C 07                                cmp AL, 7 ; F7
393    05D2 75 0A                                jne _F8

```

394	05D4	F6 16 0177r	not
translateEnabled			
395	05D8	E8 FF22	call changeFx
396	05DB	EB 0F 90	jmp
_translate_or_ignore			
397	05DE		_F8:
398	05DE	3C 08	cmp AL, 8 ; F8
399	05E0	75 0A	jne
_translate_or_ignore			

tsr.asm

400	05E2	F6 16 016Cr		not ignoreEnabled
401	05E6	E8 FF14		call changeFx
402	05E9	EB 01 90		jmp
_translate_or_ignore				
403				
404				;игнорирование и перевод
405	05EC			_translate_or_ignore:
406				
407	05EC	9C		pushf
408	05ED	2E: FF 1E 019Br		call dword ptr CS:
[old_int9hOffset] ; вызываем стандартный обработчик прерывания				
409	05F2	B8 0040	mov	AX, 40h ;
40h-сегмент, где хранятся флаги сост-я клави, кольц. +				
410				буфер ввода
411	05F5	8E C0	mov	ES, AX
412	05F7	26: 8B 1E 001C	mov	BX, ES:
[1Ch] ; адрес хвоста				
413	05FC	4B	dec	BX ; сместимся
назад к последнему				
414	05FD	4B	dec	BX ; введённому
символу				
415	05FE	83 FB 1E	cmp	BX, 1Eh ; не
вышли ли мы за пределы буфера?				
416	0601	73 03	jae	_go
417	0603	BB 003C	mov	BX, 3Ch ; хвост
вышел за пределы буфера, значит последний введённый +				
418				СИМВОЛ
419				; находится в
конце буфера				


```

445                                ;mov ES:[BX], AX
446                                ;@   на месте AX может быть
'*' для замены всех символов множества ignoredChars  +
447                                на   звёздочки
448                                ;@   или, для перевода одних
      символов в другие - завести массив
449                                ;@   replaceWith DB '...',
где перечислить символы, на которые пойдёт замена
450                                ;@   и раскомментировать
строки ниже:
451                                ;xor AX, AX
452                                ;mov AL, replaceWith[SI]
453                                ;mov ES:[BX], AX          ;
замена символа
454      0627  EB 23 90                                jmp _quit
455
456      062A                                _check_translate:

```

tsr.asm

```

457                                ; включен ли режим
    перевода?
458    062A  80 3E 0177r FF        cmp translateEnabled,
true                                true
459    062F  75 1B                jne _quit
460
461                                ; да, включен
462    0631  BE 0000                mov SI, 0
463    0634  B9 0005                mov CX,
translateLength ; кол-во символов для перевода
464                                ; проверяем, присутствует ли
текущий символ в списке для перевода
465    0637                                _check_translate_loop:
466    0637  3A 94 016Dr            cmp DL,
translateFrom[SI]
467    063B  74 06                je     _translate
468    063D  46                    inc SI
469    063E  E2 F7                loop
_check_translate_loop
470    0640  EB 0A 90                jmp _quit
471
472                                ; переводим
473    0643                                _translate:
474    0643  33 C0                xor AX, AX
475    0645  8A 84 0172r            mov AL,
translateTo[SI]
476    0649  26: 89 07                mov ES:[BX], AX ;
замена символа
477

```

```

478      064C                                _quit:
479                                           ; восстанавливаем все регистры
480      064C  1F                                pop     DS
481      064D  07                                pop     ES
482      064E  5A                                pop     DX
483      064F  59                                pop     CX
484      0650  5B                                pop     BX
485      0651  58                                pop     AX
486      0652  5E                                pop     SI
487      0653  CF                                iret
488      0654                                new_int9h endp
489
490                                           ;=== Обработчик прерывания int 1Ch
===;
491                                           ;=== Вызывается каждые 55 мс ===;
492      0654                                new_int1Ch proc far
493      0654  50                                push    AX
494      0655  0E                                push    CS
495      0656  1F                                pop     DS
496
497      0657  9C                                pushf
498      0658  2E: FF 1E 019Fr                    call dword ptr CS:
[old_int1ChOffset]
499
500      065D  80 3E 0178r FF                    cmp signaturePrintingEnabled,
true ; если нажата управляющая клавиша (в данном случае +
501                                           F6)
502      0662  75 1C                                jne _notToPrint
503
504      0664  83 3E 01A9r 5B                    cmp counter,
printDelay*1000/55 + 1 ; если кол-во "тактов" эквивалентно +
505                                           %printDelay% секундам
506      0669  74 03                                je      _letsPrint

```

```
507
508    066B  EB 0E 90                                jmp _dontPrint
509
510    066E                                _letsPrint:
511    066E  F6 16 0178r                            not
signaturePrintingEnabled
512    0672  C7 06 01A9r 0000                        mov counter,
0
513    0678  E8 0094                                call printSignature
```

tsr.asm

```

514
515      067B                                _dontPrint:
516      067B  83 06 01A9r 01                add counter, 1
517
518      0680                                _notToPrint:
519
520      0680  58                            pop AX
521
522      0681  CF                            iret
523      0682                                new_int1Ch endp
524
525                                ;=== Обработчик прерывания int 2Fh
    ===;
526                                ;=== Служит для:
527                                ;=== 1) проверки факта присутствия TSR
в памяти (при AH=0FFh, AL=0)
528                                ;===      будет возвращён AH='i' в
случае, если TSR уже загружен
529                                ;=== 2) выгрузки TSR из памяти (при
AH=0FFh, AL=1)
530                                ;===
531      0682                                new_int2Fh proc
532      0682  80 FC FF                        cmp     AH, 0FFh      ;наша
функция?
533      0685  75 0B                        jne     _2Fh_std      ;нет -
на старый обработчик
534      0687  3C 00                        cmp     AL, 0      ;подфункция
проверки, загружен ли резидент в память?
535      0689  74 0C                        je      _already_installed

```

```

536  068B 3C 01                cmp     AL, 1    ;подфункция
выгрузки из памяти?
537  068D 74 0B                je      _uninstall
538  068F EB 01 90             jmp     _2Fh_std    ;нет -
на старый обработчик
539
540  0692                    _2Fh_std:
541  0692 2E: FF 2E 01A3r      jmp     dword ptr CS:
[old_int2Fh0ffset] ;вызов старого обработчика
542
543  0697                    _already_installed:
544  0697 B4 69                mov     AH, 'i' ;вернём
'i', если резидент загружен в память
545  0699 CF                    iret
546
547  069A                    _uninstall:
548  069A 1E                    push    DS
549  069B 06                    push    ES
550  069C 52                    push    DX
551  069D 53                    push    BX
552
553  069E 33 DB                xor     BX, BX
554
555                                ; CS = ES, для доступа к переменным
556  06A0 0E                    push    CS
557  06A1 07                    pop     ES
558
559  06A2 B8 2509              mov     AX, 2509h
560  06A5 26: 8B 16 019Br      mov     DX, ES:old_int9h0ffset
; возвращаем вектор прерывания
561  06AA 26: 8E 1E 019Dr      mov     DS, ES:old_int9hSegment
; на место
562  06AF CD 21                int     21h

```



```

563
564    06B1  B8 251C                mov     AX, 251Ch
565    06B4  26: 8B 16 019Fr        mov DX, ES:old_int1ChOffset ;
возвращаем вектор прерывания
566    06B9  26: 8E 1E 01A1r        mov DS, ES:old_int1ChSegment
; на место
567    06BE  CD 21                int     21h
568
569    06C0  B8 252F                mov     AX, 252Fh
570    06C3  26: 8B 16 01A3r        mov DX, ES:old_int2FhOffset ;
возвращаем вектор прерывания

```

tsr.asm

```

    571    06C8  26: 8E 1E 01A5r      mov DS, ES:old_int2FhSegment
; на место
    572    06CD  CD 21                int     21h
    573
    574    06CF  2E: 8E 06 002C      mov     ES, CS:2Ch          ;
загрузим в ES адрес окружения
    575    06D4  B4 49                mov     AH, 49h          ;
выгрузим из памяти окружение
    576    06D6  CD 21                int     21h
    577    06D8  72 0B                jc      _notRemove
    578
    579    06DA  0E                    push    CS
    580    06DB  07                    pop     ES          ;в ES - адрес
резидентной программы
    581    06DC  B4 49                mov     AH, 49h          ;выгрузим из
памяти резидент
    582    06DE  CD 21                int     21h
    583    06E0  72 03                jc      _notRemove
    584    06E2  EB 15 90              jmp     _unloaded
    585
    586    06E5                        _notRemove: ; не удалось выполнить
выгрузку
    587
; вывод сообщения о неудачной выгрузке
    588    06E5  B4 03                mov     AH, 03h
; получаем позицию курсора
    589    06E7  CD 10                int     10h
    590    06E9  BD 04D8r             lea     BP, noRemoveMsg
    591    06EC  B9 001D              mov     CX, noRemoveMsg_length
    592    06EF  B3 07                mov     BL, 0111b

```

```

593    06F1  B8 1301                mov AX, 1301h
594    06F4  CD 10                  int 10h
595    06F6  EB 12 90                jmp _2Fh_exit
596
597    06F9                                _unloaded: ; выгрузка прошла успешно
598                                ; вывод сообщения об удачной выгрузке
599    06F9  B4 03                mov AH, 03h
; получаем позицию курсора
600    06FB  CD 10                int 10h
601    06FD  BD 04C7r             lea BP, removedMsg
602    0700  B9 0011             mov CX, removedMsg_length
603    0703  B3 07                mov BL, 0111b
604    0705  B8 1301             mov AX, 1301h
605    0708  CD 10                int 10h
606
607    070A                                _2Fh_exit:
608    070A  5B                    pop BX
609    070B  5A                    pop  DX
610    070C  07                    pop  ES
611    070D  1F                    pop  DS
612    070E  CF                    iret
613    070F                                new_int2Fh endp
614
615                                ;== Процедура вывода подписи (ФИО,
группа)
616                                ;== Настраивается значениями
переменных в начале исходника
617                                ;==
618    070F                                printSignature proc
619    070F  50                    push AX
620    0710  52                    push DX
621    0711  51                    push CX

```

622	0712	53	push BX
623	0713	06	push ES
624	0714	54	push SP
625	0715	55	push BP
626	0716	56	push SI
627	0717	57	push DI

tsr.asm

```

628
629 0718 33 C0                xor AX, AX
630 071A 33 DB                xor BX, BX
631 071C 33 D2                xor DX, DX
632
633 071E B4 03                mov AH, 03h
    ; чтение текущей позиции курсора
634 0720 CD 10                int 10h
635 0722 52                    push DX
    ; помещаем информацию о      +
636                                положении курсора в стек
637
638 0723 83 3E 01ABr 00        cmp printPos, 0
639 0728 74 0E                je  _printTop
640
641 072A 83 3E 01ABr 01        cmp printPos, 1
642 072F 74 0E                je  _printCenter
643
644 0731 83 3E 01ABr 02        cmp printPos, 2
645 0736 74 0E                je  _printBottom
646
647                                ; все числа подобраны на глаз...
648 0738                        _printTop:
649 0738 B6 00                mov DH, 0
650 073A B2 0F                mov DL, 15
651 073C EB 0F 90            jmp _actualPrint
652

```

```

653      073F                      _printCenter:
654      073F  B6 09                      mov DH, 9
655      0741  B2 0F                      mov DL, 15
656      0743  EB 08 90                  jmp _actualPrint
657
658      0746                      _printBottom:
659      0746  B6 13                      mov DH, 19
660      0748  B2 0F                      mov DL, 15
661      074A  EB 01 90                  jmp _actualPrint
662
663      074D                      _actualPrint:
664      074D  B4 0F                      mov AH, 0Fh
        ; чтение текущего видеорежима. в+
665                        BH      - текущая страница
666      074F  CD 10                      int 10h
667
668      0751  0E                      push CS
669      0752  07                      pop ES
        ; указываем ES на CS
670
671                        ; вывод 'верхушки' таблицы
672      0753  52                      push DX
673      0754  BD 03FAr                  lea BP, tableTop
        ; помещаем в BP указатель на +
674                        выводимую строку
675      0757  B9 0039                  mov CX, tableTop_length
        ; в CX - длина строки
676      075A  B3 07                      mov BL, 0111b
        ; цвет выводимого текста ref: +
677
http://en.wikipedia.org/wiki/BIOS\_color\_attributes
678      075C  B8 1301                  mov AX, 1301h
        ; AH=13h - номер ф-ии, AL=01h - +

```

679		
из	символов	строки
680	075F	CD 10
681	0761	5A
682	0762	FE C6
683		
684		

курсор перемещается при выводе каждого

int 10h

pop DX

inc DH

tsr.asm

```
685                                ;вывод первой линии
686    0764  52                    push DX
687    0765  BD 01ADr              lea BP, signatureLine1
688    0768  B9 0039              mov CX, Line1_length
689    076B  B3 07                mov BL, 0111b
690    076D  B8 1301              mov AX, 1301h
691    0770  CD 10                int 10h
692    0772  5A                    pop DX
693    0773  FE C6                inc DH
694
695                                ;вывод второй линии
696    0775  52                    push DX
697    0776  BD 01E6r              lea BP, signatureLine2
698    0779  B9 0039              mov CX, Line2_length
699    077C  B3 07                mov BL, 0111b
700    077E  B8 1301              mov AX, 1301h
701    0781  CD 10                int 10h
702    0783  5A                    pop DX
703    0784  FE C6                inc DH
704
705                                ;вывод третьей линии
706    0786  52                    push DX
707    0787  BD 021Fr              lea BP, signatureLine3
708    078A  B9 0039              mov CX, Line3_length
709    078D  B3 07                mov BL, 0111b
710    078F  B8 1301              mov AX, 1301h
```



```

711    0792  CD 10                                int 10h
712    0794  5A                                pop DX
713    0795  FE C6                            inc DH
714
715                                           ;вывод 'низа' таблицы
716    0797  52                                push DX
717    0798  BD 0433r                          lea BP, tableBottom
718    079B  B9 0039                          mov CX,
tableBottom_length
719    079E  B3 07                            mov BL, 0111b
720    07A0  B8 1301                          mov AX, 1301h
721    07A3  CD 10                                int 10h
722    07A5  5A                                pop DX
723    07A6  FE C6                            inc DH
724
725    07A8  33 DB                            xor BX, BX
726    07AA  5A                                pop DX
      ;восстанавливаем из стека      +
727                                           прежнее положение курсора
728    07AB  B4 02                                mov AH, 02h
      ;меняем положение курсора на  +
729                                           первоначальное
730    07AD  CD 10                                int 10h
731    07AF  E8 FD4B                          call changeFx
732
733    07B2  5F                                pop DI
734    07B3  5E                                pop SI
735    07B4  5D                                pop BP
736    07B5  5C                                pop SP
737    07B6  07                                pop ES
738    07B7  5B                                pop BX
739    07B8  59                                pop CX

```

740	07B9	5A	pop DX
741	07BA	58	pop AX

tsr.asm

```

742
743     07BB  C3                      ret
744     07BC                      printSignature endp
745
746                                     ;== Функция, которая в зависимости от
флага cursiveEnabled меняет начертание символа с курсива+
747                                     на обычное и наоборот
748                                     ;== Сама смена происходит в процедуре
changeFont, а здесь подготавливаются данные
749     07BC                      setCursive proc
750     07BC  06                      push ES ; сохраняем регистры
751     07BD  50                      push AX
752     07BE  0E                      push CS
753     07BF  07                      pop ES
754
755     07C0  80 3E 0179r FF          cmp cursiveEnabled, true
756     07C5  75 30                      jne _restoreSymbol
757                                     ; если флаг равен true, выполняем
замену символа на курсивный вариант,
758                                     ; предварительно сохраняя старый
символ в savedSymbol
759
760     07C7  E8 004C                      call saveFont
761     07CA  8A 0E 018Ar              mov CL, charToCursiveIndex
762     07CE                      _shiftTable:
763                                     ; мы получаем в BP таблицу
всех символов. адрес указывает на символ 0

```

764					; поэтому нуэно совершить сдвиг
16*X	- где X	- код		символа	
765	07CE	83	C5	10	add BP, 16
766	07D1	E2	FB		loop _shiftTable
767					
768					; при savefont смещается регистр ES
769					; поэтому приходится делать такие
махинации, чтобы					
770					; записать полученный элемент в
savedSymbol					
771					; swap(ES, DS) и сохранение старого
значения DS					
772	07D3	1E			push DS
773	07D4	58			pop AX
774	07D5	06			push ES
775	07D6	1F			pop DS
776	07D7	50			push AX
777	07D8	07			pop ES
778	07D9	50			push AX
779					
780	07DA	8B	F5		mov SI, BP
781	07DC	BF	018Br		lea DI, savedSymbol
782					; сохраняем в переменную
savedSymbol					
783					; таблицу нужного символа
784	07DF	B9	0010		mov CX, 16
785					; movsb из DS:SI в ES:DI
786	07E2	F3>	A4		rep movsb
787					; исходные позиции сегментов
возвращены					
788	07E4	1F			pop DS ; восстановление DS
789					
790					; заменим написание символа на
курсив					

```
791    07E5  B9 0001                mov CX, 1
792    07E8  B6 00                mov DH, 0
793    07EA  8A 16 018Ar          mov DL, charToCursiveIndex
794    07EE  BD 017Ar          lea BP, cursiveSymbol
795    07F1  E8 0015          call changeFont
796    07F4  EB 10 90          jmp _exitSetCursive
797
798    07F7                _restoreSymbol:
```

tsr.asm

```

799                                     ; если флаг равен 0, выполняем
замену курсивного символа на старый вариант
800
801    07F7  B9 0001                    mov CX, 1
802    07FA  B6 00                      mov DH, 0
803    07FC  8A 16 018Ar                mov DL, charToCursiveIndex
804    0800  BD 018Br                    lea bp, savedSymbol
805    0803  E8 0003                    call changeFont
806
807    0806                                _exitSetCursive:
808    0806  58                          pop AX
809    0807  07                          pop ES
810    0808  C3                          ret
811    0809                                setCursive endp
812
813                                     ;=== Функция смены начертания символа
(курсив/нормальное)
814                                     ;===
815                                     ; *** входные данные
816                                     ; DL = номер символа для замены
817                                     ; CX = Кол-во символов заменяемых
изображений символов
818                                     ; (начиная с символа указанного в DX)
819                                     ; ES:bp = адрес таблицы
820                                     ;
821                                     ; *** описание работы процедуры
822                                     ; Происходит вызов int 10h
(видеосервис)

```

823		; с функцией AH = 11h (функции
знакогенератора)		
824		; Параметр AL = 0 сообщает, что будет
заменено изображение		
825		; символа для текущего шрифта
826		; В случаях, когда AL = 1 или 2,
будет заменено изображение		
827		; только для определенного шрифта (8x14 и
8x8 соответственно)		
828		; Параметр BH = 0Eh сообщает, что на
определение каждого изображения символа		
829		; расходуется по 14 байт (режим 8x14 бит
как раз 14 байт)		
830		; Параметр BL = 0 - блок шрифта для
загрузки (от 0 до 4)		
831		;
832		; *** результат
833		; изображение указанного(ых) символа(ов)
будет заменено		
834		; на предложенное пользователем.
835		; Изменению подвергнутся все символы,
находящиеся на экране,		
836		; то есть если изображение заменено,
старый вариант нигде уже не проявится		
837		
838	0809	changeFont proc
839	0809 50	push AX
840	080A 53	push BX
841	080B B8 1100	mov AX, 1100h
842	080E BB 1000	mov BX, 1000h
843	0811 CD 10	int 10h
844	0813 58	pop AX
845	0814 5B	pop BX
846	0815 C3	ret
847	0816	changeFont endp

848	
849	;== Функция сохранения нормального
начертания символа	
850	;==
851	; *** входные данные
852	; ВН - тип возвращаемой символьной
таблицы	
853	; 0 - таблица из int 1fh
854	; 1 - таблица из int 44h
855	; 2-5 - таблица из 8x14, 8x8,
8x8 (top), 9x14	

tsr.asm

```

856                ;      6 - 8x16
857                ;
858                ; *** описание работы процедуры
859                ; Происходит вызов      int 10h
      (видеосервис)
860                ; с функцией AH = 11h (функции
знакогенератора)
861                ; Параметр AL = 30      - подфункция
получения информации о EGA
862                ;
863                ; *** результат
864                ; в ES:BP находится таблица символов
(полная)
865                ; в CX находится байт на символ
866                ; в DL количество экранных строк
867                ; ВАЖНО! Происходит сдвиг регистра      ES
868                ; ( ES становится равным C000h )
869
870      0816        saveFont proc
871      0816  50          push AX
872      0817  53          push BX
873      0818  B8 1130      mov AX, 1130h
874      081B  BB 0600      mov BX, 0600h
875      081E  CD 10        int 10h
876      0820  58          pop AX
877      0821  5B          pop BX
878      0822  C3          ret
879      0823        saveFont endp

```

```

880
881
882                ;=== Отсюда начинается выполнение
основной части программы ===;
883                ;===
884    0823          _initTSR: ; старт резидента
885    0823 B4 03             mov AH, 03h
886    0825 CD 10            int 10h
887    0827 52              push DX
888    0828 B4 00             mov AH,00h ; установка
видеорежима      (83h текст 80x25 16/8 CGA,EGA b800 Comp,RGB, +
889                  Enhanced), без очистки экрана
890    082A B0 83             mov AL,83h
891    082C CD 10            int 10h
892    082E 5A              pop DX
893    082F B4 02             mov AH, 02h
894    0831 CD 10            int 10h
895
896
897    0833 E8 0095           call commandParamsParser
898    0836 B8 3509           mov AX,3509h                      ;
получить в ES:BX вектор 09
899    0839 CD 21            int 21h                        ; прерывания
900
901                ;@     == Удаление резидента из
памяти ==
902                ;@     Если по варианту
необходимо выгружать резидент по повторному запуску приложений,
903                ;@     нужно закомментировать
следующие 3 строки, а также
904                ;@     содержимое метки _finishTSR
ф-ии commandParamsParser, но не саму метку!
905                ;cmp unloadTSR, true
906                ;je removingOnParameter
```

```
907                                     ;jmp _notRemovingNow
908
909    083B                               _removingOnParameter:
910    083B B4 FF                         mov AH, 0FFh
911    083D B0 00                         mov AL, 0
912    083F CD 2F                         int 2Fh
```

tsr.asm

```

    913    0841  80 FC 69                      cmp AH, 'i' ; проверка
того, загружена ли уже программа
    914    0844  74 69                      je    _remove
    915                                         ;mov AH, 09h
;@ для выгрузки резидента по повторному+
    916                                запуску закомментировать эту строку
    917                                ;lea DX, notInstalledMsg
;@ для выгрузки резидента по повторному запуску+
    918                                закомментировать эту строку
    919                                ;int 21h
        ;@ для выгрузки резидента по +
    920                                повторному запуску закомментировать
эту строку
    921                                ;int 20h
        ;@ для выгрузки резидента по +
    922                                повторному запуску закомментировать
эту строку
    923
    924    0846                                _notRemovingNow:
    925
    926    0846  80 3E 01A8r FF                cmp notLoadTSR, true
; если была выведена справка
    927    084B  74 03                      je    _exit_tmp
        ; просто выходим
    928
    929                                ;@ Если по варианту
необходимо выгружать резидент по повторному запуску, то +
    930                                комментируем 5 строк ниже
    931                                ;@ если необходимо выгружать по
параметру командной строки, то оставляем их

```

```

932                                ;mov AH, 0FFh
933                                ;mov AL, 0
934                                ;int 2Fh
935                                ;cmp AH, 'i' ; проверка того,
загружена ли уже программа
936                                ;je _alreadyInstalled
937
938    084D EB 04 90                jmp _tmp
939
940    0850                        _exit_tmp:
941    0850 EB 77 90                jmp _exit
942
943    0853                        _tmp:
944    0853 06                    push ES
945    0854 A1 002C                mov AX, DS:[2Ch] ; psp
946    0857 8E C0                mov ES, AX
947    0859 B4 49                mov AH, 49h ;
хватит памяти чтоб остаться
948    085B CD 21                int 21h ;
резидентом?
949    085D 07                    pop ES
950    085E 72 62                jc _notMem ; не
хватило - выходим
951
952                                ;== int 09h ==;
953
954    0860 2E: 89 1E 019Br        mov word ptr
CS:old_int9hOffset, BX
955    0865 2E: 8C 06 019Dr        mov word ptr
CS:old_int9hSegment, ES
956    086A B8 2509                mov AX, 2509h ;
установим вектор на 09
957    086D BA 059Fr              mov DX, offset new_int9h ;
прерывание

```

```

958      0870  CD 21                      int 21h
959
960                                     ;== int 1Ch ==;
961      0872  B8 351C                      mov AX,351Ch                ;
получить в ES:BX      вектор 1C
962      0875  CD 21                      int 21h                ; прерывания
963      0877  2E: 89 1E 019Fr            mov      word ptr
CS:old_int1ChOffset, BX
964      087C  2E: 8C 06 01A1r            mov      word ptr
CS:old_int1ChSegment, ES
965      0881  B8 251C                      mov AX, 251Ch                ;
установим вектор на 1C
966      0884  BA 0654r                    mov DX, offset new_int1Ch
      ; прерывание
967      0887  CD 21                      int 21h
968
969                                     ;== int 2Fh ==;

```

tsr.asm

```

    970    0889  B8 352F                mov AX,352Fh                ;
получить  в ES:BX    вектор 1C

    971    088C  CD 21                int 21h                ; прерывания

    972    088E  2E: 89 1E 01A3r      mov     word ptr
CS:old_int2FhOffset, BX

    973    0893  2E: 8C 06 01A5r      mov     word ptr
CS:old_int2FhSegment, ES

    974    0898  B8 252F                mov AX, 252Fh                ;
установим вектор на 2F

    975    089B  BA 0682r             mov DX, offset new_int2Fh
        ; прерывание

    976    089E  CD 21                int 21h

    977

    978    08A0  E8 FC5A                call changeFx

    979    08A3  BA 046Cr             mov DX, offset installedMsg
; выводим что все ОК

    980    08A6  B4 09                mov AH, 9

    981    08A8  CD 21                int 21h

    982    08AA  BA 0823r             mov DX, offset _initTSR      ;
остаемся в памяти резидентом

    983    08AD  CD 27                int 27h                ; и выходим

    984                                ; конец основной программы

    985    08AF                        _remove: ; выгрузка программы из памяти

    986    08AF  B4 FF                mov AH, 0FFh

    987    08B1  B0 01                mov AL, 1

    988    08B3  CD 2F                int 2Fh

    989    08B5  EB 12 90                jmp _exit

    990    08B8                        _alreadyInstalled:

    991    08B8  B4 09                mov AH, 09h

```

```

992    08BA  BA 047Fr          lea DX, alreadyInstalledMsg
993    08BD  CD 21             int 21h
994    08BF  EB 08 90          jmp _exit
995    08C2                                _notMem:          ; не хватает
памяти, чтобы остаться резидентом
996    08C2  BA 0495r          mov DX, offset noMemMsg
997    08C5  B4 09             mov AH, 9
998    08C7  CD 21             int 21h
999    08C9                                _exit:              ; выход
1000   08C9  CD 20             int 20h
1001
1002                                ;== Процедура проверки параметров
ком. строки ==;
1003                                ;==
1004   08CB                                commandParamsParser proc
1005   08CB  0E                  push CS
1006   08CC  07                  pop ES
1007   08CD  C6 06 01A7r 00      mov unloadTSR, 0
1008   08D2  C6 06 01A8r 00      mov notLoadTSR, 0
1009
1010   08D7  BE 0080              mov SI, 80h
;SI=смещение командной строки.
1011   08DA  AC                  lodsb
;Получим кол-во символов.
1012   08DB  0A C0              or    AL, AL
;Если 0 символов введено,
1013   08DD  74 40              jz    _exitHelp
;то все в порядке.
1014
1015   08DF                                _nextChar:
1016
1017   08DF  46                  inc SI
;Теперь SI указывает на первый символ +
1018                                строки.

```



```

1019
1020  08E0  80 3C 0D          cmp [SI], BYTE ptr 13
1021  08E3  74 3A          je  _exitHelp
1022
1023
1024  08E5  AD          lodsw
      ;Получаем два символа
1025  08E6  3D 3F2F          cmp AX, '?'
      ;Это '/' (данные расположены в +
1026                                обратном порядке, т.е. AL:AH вместо
      AH:AL)

```

tsr.asm

```

1027    08E9  74 03                                je    _question
1028                                           ;cmp AX, 'u/'
1029                                           ;je _finishTSR
1030
1031                                           ;cmp AH, '/'
1032                                           ;je _errorParam
1033
1034    08EB  EB 32 90                                jmp _exitHelp
1035
1036    08EE                                _question:
1037                                           ; вывод строки помощи
1038    08EE  B4 03                                mov AH,03
1039    08F0  CD 10                                int 10h
1040    08F2  BD 0258r                             lea BP, helpMsg
1041    08F5  B9 017D                             mov CX,
helpMsg_length
1042    08F8  B3 07                                mov BL, 0111b
1043    08FA  B8 1301                             mov AX, 1301h
1044    08FD  CD 10                                int 10h
1045                                           ; конец вывода строки помощи
1046    08FF  F6 16 01A8r                         not notLoadTSR
;флаг того, что необходимо не загружать резидент
1047    0903  EB DA                                jmp _nextChar
1048
1049                                           ;@    === Удаление резидента из
памяти ===
1050                                           ;@    Если по    варианту
необходимо выгружать резидент по параметру '/u' командной строки,

```

```

1051                                     ;@    нужно использовать следующий
код, в остальных случаях необходимо закоментировать
1052                                     ;@    этот код, кроме названия
метки!    (по желанию можно избавиться и от метки, но    +
1053                                     аккуратно просмотреть использование)
1054    0905                                _finishTSR:
1055    0905  F6 16 01A7r                    not unloadTSR
;флаг того, что необходимо выгрузить резидент
1056    0909  EB D4                        jmp _nextChar
1057
1058    090B  EB 12 90                        jmp _exitHelp
1059
1060    090E                                _errorParam:
1061                                     ;вывод строки
1062    090E  B4 03                            mov AH,03
1063    0910  CD 10                            int 10h
1064    0912  BD 03D5r                        lea BP,
CS:errorParamMsg
1065    0915  B9 0025                        mov CX,
errorParamMsg_length
1066    0918  B3 07                            mov BL, 0111b
1067    091A  B8 1301                        mov AX, 1301h
1068    091D  CD 10                            int 10h
1069                                     ;конец вывода строки
1070    091F                                _exitHelp:
1071    091F  C3                            ret
1072    0920                                commandParamsParser endp
1073
1074    0920                                code ends
1075                                end _start

```

Symbol Table

Symbol Name	Type	Value
??DATE	Text	"05/09/26"
??FILENAME	Text	"tsr "
??TIME	Text	"22:53:14"
??VERSION	Number	030A
@CPU	Text	0101H
@CURSEG	Text	CODE
@FILENAME	Text	TSR
@WORDSIZE	Text	2
ALREADYINSTALLEDMSG	Byte	CODE:047F
CHANGEFONT	Near	CODE:0809
CHANGEFX	Near	CODE:04FD
CHARTOCURSIVEINDEX	Byte	CODE:018A
COMMANDPARAMSPARSER	Near	CODE:08CB
COUNTER	Word	CODE:01A9
CURSIVEENABLED	Byte	CODE:0179
CURSIVESYMBOL	Byte	CODE:017A
ERRORPARAMMSG	Byte	CODE:03D5
ERRORPARAMMSG_LENGTH	Number	0025
F1_TXT	Byte	CODE:04F5
F2_TXT	Byte	CODE:04F7
F3_TXT	Byte	CODE:04F9
F4_TXT	Byte	CODE:04FB
FX_LENGTH	Number	0002

HELPMMSG	Byte	CODE:0258
HELPMMSG_LENGTH	Number	017D
IGNOREDCHARS	Byte	CODE:0103
IGNOREDLENGTH	Number	0069
IGNOREENABLED	Byte	CODE:016C
INSTALLEDMSG	Byte	CODE:046C
LINE1_LENGTH	Number	0039
LINE2_LENGTH	Number	0039
LINE3_LENGTH	Number	0039
NEW_INT1CH	Far	CODE:0654
NEW_INT2FH	Near	CODE:0682
NEW_INT9H	Far	CODE:059F
NOMEMMSG	Byte	CODE:0495
NOREMOVEMSG	Byte	CODE:04D8
NOREMOVEMSG_LENGTH	Number	001D
NOTINSTALLEDMSG	Byte	CODE:04A9
NOTLOADTSR	Byte	CODE:01A8
OLD_INT1CH0FFSET	Word	CODE:019F
OLD_INT1CHSEGMENT	Word	CODE:01A1
OLD_INT2FH0FFSET	Word	CODE:01A3
OLD_INT2FHSEGMENT	Word	CODE:01A5
OLD_INT9H0FFSET	Word	CODE:019B
OLD_INT9HSEGMENT	Word	CODE:019D
PRINTDELAY	Number	0005
PRINTPOS	Word	CODE:01AB
PRINTSIGNATURE	Near	CODE:070F
REMOVEDMSG	Byte	CODE:04C7
REMOVEDMSG_LENGTH	Number	0011
SAVEDSYMBOL	Byte	CODE:018B
SAVEFONT	Near	CODE:0816
SETCURSIVE	Near	CODE:07BC

Symbol Table

SIGNATURELINE1	Byte	CODE:01AD
SIGNATURELINE2	Byte	CODE:01E6
SIGNATURELINE3	Byte	CODE:021F
SIGNATUREPRINTINGENABLED	Byte	CODE:0178
TABLEBOTTOM	Byte	CODE:0433
TABLEBOTTOM_LENGTH	Number	0039
TABLETOP	Byte	CODE:03FA
TABLETOP_LENGTH	Number	0039
TRANSLATEENABLED	Byte	CODE:0177
TRANSLATEFROM	Byte	CODE:016D
TRANSLATELENGTH	Number	0005
TRANSLATETO	Byte	CODE:0172
TRUE	Number	00FF
UNLOADTSR	Byte	CODE:01A7
_2FH_EXIT	Near	CODE:070A
_2FH_STD	Near	CODE:0692
_ACTUALPRINT	Near	CODE:074D
_ALREADYINSTALLED	Near	CODE:08B8
_ALREADY_INSTALLED	Near	CODE:0697
_BLOCK	Near	CODE:0622
_CHECKF5	Near	CODE:050C
_CHECKF6	Near	CODE:0530
_CHECKF7	Near	CODE:0551
_CHECKF8	Near	CODE:0572
_CHECK_IGNORED	Near	CODE:0616
_CHECK_TRANSLATE	Near	CODE:062A

_CHECK_TRANSLATE_LOOP	Near	CODE:0637
_DONTPRINT	Near	CODE:067B
_ERRORPARAM	Near	CODE:090E
_EXIT	Near	CODE:08C9
_EXITHELP	Near	CODE:091F
_EXITSETCURSIVE	Near	CODE:0806
_EXIT_TMP	Near	CODE:0850
_F5	Near	CODE:05B1
_F6	Near	CODE:05BF
_F7	Near	CODE:05D0
_F8	Near	CODE:05DE
_FINISHTSR	Near	CODE:0905
_G0	Near	CODE:0606
_GREENF5	Near	CODE:0529
_GREENF6	Near	CODE:054D
_GREENF7	Near	CODE:056E
_GREENF8	Near	CODE:058F
_INITTSR	Near	CODE:0823
_LETSPRINT	Near	CODE:066E
_NEXTCHAR	Near	CODE:08DF
_NOTMEM	Near	CODE:08C2
_NOTREMOVE	Near	CODE:06E5
_NOTREMOVINGNOW	Near	CODE:0846
_NOTTOPRINT	Near	CODE:0680
_OUTFX	Near	CODE:0593
_PRINTBOTTOM	Near	CODE:0746
_PRINTCENTER	Near	CODE:073F
_PRINTTOP	Near	CODE:0738
_QUESTION	Near	CODE:08EE
_QUIT	Near	CODE:064C
_REDF5	Near	CODE:0522

Symbol Table

_REDF6	Near	CODE:0546
_REDF7	Near	CODE:0567
_REDF8	Near	CODE:0588
_REMOVE	Near	CODE:08AF
_REMOVINGONPARAMETER	Near	CODE:083B
_RESTORESymbOL	Near	CODE:07F7
_SHIFTTABLE	Near	CODE:07CE
_START	Near	CODE:0100
_TEST_FX	Near	CODE:05AF
_TMP	Near	CODE:0853
_TRANSLATE	Near	CODE:0643
_TRANSLATE_OR_IGNORE	Near	CODE:05EC
_UNINSTALL	Near	CODE:069A
_UNLOADED	Near	CODE:06F9

Groups & Segments

Bit Size Align Combine Class

CODE	16	0920	Para	none	CODE
------	----	------	------	------	------